MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 16 — Héritage et Polymorphisme

Hériter, spécialiser, polymorphiser — la POO avancée

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

14 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

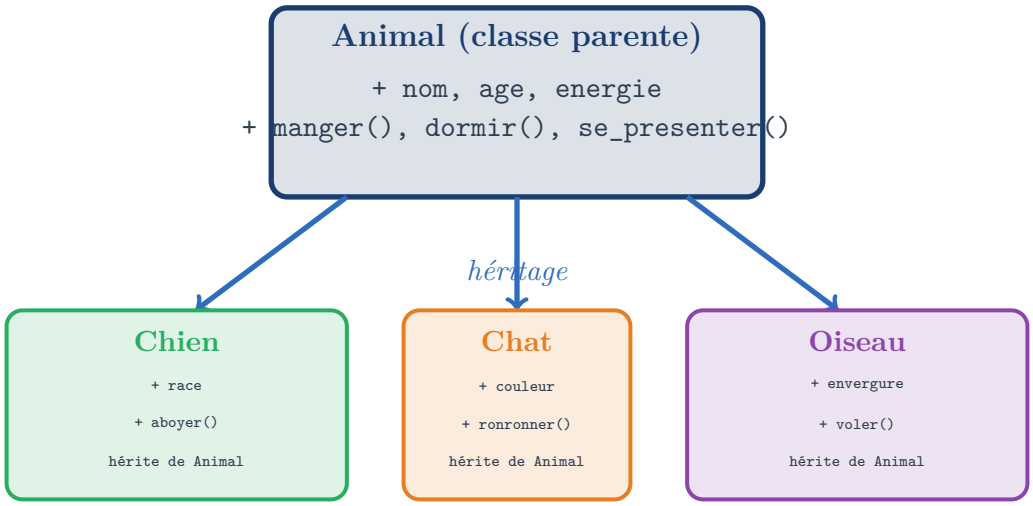
Chapitre 16 — Héritage et Polymorphisme **2**
0.1 L'héritage simple 2
0.2 super() — Appeler la classe parente 4
0.3 Le polymorphisme 5
0.4 Les classes abstraites 7
0.5 L'héritage multiple 8
0.6 Les propriétés — @property 9
0.7 @staticmethod et @classmethod 11
0.8 Ce qui m'a le plus surpris dans ce chapitre 12
0.9 Résumé du Chapitre 16 12

Chapitre 16 — Héritage et Polymorphisme

À retenir

L'**héritage** permet à une classe de **réutiliser** le code d'une autre classe en l'étendant ou en le spécialisant. Le **polymorphisme** permet à des objets de classes différentes de répondre à la même interface de façon unique. Ces deux mécanismes sont le cœur de la POO — ils permettent de construire des architectures flexibles, extensibles et sans duplication de code.

L'héritage simple



Héritage simple — classe Animal

```python
class Animal:
    """Classe parente     Animal generique."""

    def __init__(self, nom, age):
        self.nom      = nom
        self.age      = age
        self.energie  = 100

    def manger(self):
        self.energie += 20
        print(f"{self.nom} mange. Energie : {self.energie}")

    def dormir(self):
        self.energie += 50
        print(f"{self.nom} dort. Energie : {self.energie}")

    def se_presenter(self):
```

```python
        print(f"Je suis {self.nom}, j'ai {self.age} ans.")

    def __str__(self):
        return f"{self.__class__.__name__}({self.nom}, {self.age} ans)"


# Classe enfant     herite de Animal
class Chien(Animal):
    """Classe Chien     herite de Animal."""

    def __init__(self, nom, age, race):
        super().__init__(nom, age)   # appel du constructeur parent
        self.race = race

    def aboyer(self):
        self.energie -= 10
        print(f"{self.nom} : Wouf ! Energie : {self.energie}")

    # Surcharge de methode (override)
    def se_presenter(self):
        super().se_presenter()   # appel de la methode parent
        print(f"Ma race : {self.race}")


class Chat(Animal):
    def __init__(self, nom, age, couleur):
        super().__init__(nom, age)
        self.couleur = couleur

    def ronronner(self):
        self.energie -= 5
        print(f"{self.nom} ronronne... Energie : {self.energie}")

    def se_presenter(self):
        super().se_presenter()
        print(f"Couleur : {self.couleur}")


# Utilisation
rex    = Chien("Rex", 3, "Berger Allemand")
misty  = Chat("Misty", 5, "gris tigre")

rex.se_presenter()
rex.manger()        # methode heritee de Animal !
rex.aboyer()

print()
```

```python

misty.se_presenter()
misty.dormir()       # methode heritee de Animal !
misty.ronronner()

print(rex)     # Chien(Rex, 3 ans)
print(misty)   # Chat(Misty, 5 ans)
```

super() — Appeler la classe parente

super() — accéder à la classe parente

```python
class Employe:
    def __init__(self, nom, salaire):
        self.nom     = nom
        self.salaire = salaire

    def travailler(self):
        print(f"{self.nom} travaille.")

    def afficher(self):
        print(f"{self.nom}     Salaire : {self.salaire}   ")


class Manager(Employe):
    def __init__(self, nom, salaire, equipe):
        # super() appelle __init__ d'Employe
        super().__init__(nom, salaire)
        self.equipe        = equipe
        self.prime         = salaire * 0.2

    def travailler(self):
        # Etendre le comportement parent
        super().travailler()
        print(f"{self.nom} manage aussi son equipe de {len(self
            .equipe)} personnes.")

    def afficher(self):
        super().afficher()
        print(f"  Prime   : {self.prime:.2f}   ")
        print(f"  Equipe  : {', '.join(self.equipe)}")


class DirecteurGeneral(Manager):
    def __init__(self, nom, salaire, equipe, entreprise):
        super().__init__(nom, salaire, equipe)
        self.entreprise = entreprise

```

```
    def afficher(self):
        super().afficher()
        print(f"  Entreprise : {self.entreprise}")


# Cha8ne d'heritage
alice = Employe("Alice", 2500)
bob   = Manager("Bob", 4000, ["Alice", "Charlie", "Diana"])
eve   = DirecteurGeneral("Eve", 8000, ["Bob", "Frank"], "
   TechCorp")

alice.afficher()
print()
bob.afficher()
print()
eve.afficher()
```

À retenir

`super()` est le mécanisme qui permet d'appeler les méthodes de la **classe parente**. Il est indispensable dans `__init__` pour initialiser correctement la partie héritée de l'objet. Sans `super().__init__()`, les attributs du parent ne seraient pas créés — l'objet serait incomplet.

Le polymorphisme

Le **polymorphisme** signifie littéralement "plusieurs formes". En POO, c'est la capacité d'objets de classes différentes à répondre à la même méthode de façon spécifique.

Polymorphisme — même interface, comportements différents

```
class Forme:
    def aire(self):
        raise NotImplementedError("Chaque forme doit
           implementer aire()")

    def perimetre(self):
        raise NotImplementedError("Chaque forme doit
           implementer perimetre()")

    def decrire(self):
        print(f"{self.__class__.__name__:15}     "
              f"Aire : {self.aire():8.2f} cm   | "
              f"P rim tre : {self.perimetre():.2f} cm")


import math
```

```python
class Cercle(Forme):
    def __init__(self, rayon):
        self.rayon = rayon

    def aire(self):
        return math.pi * self.rayon ** 2

    def perimetre(self):
        return 2 * math.pi * self.rayon


class Rectangle(Forme):
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur  = largeur

    def aire(self):
        return self.longueur * self.largeur

    def perimetre(self):
        return 2 * (self.longueur + self.largeur)


class Triangle(Forme):
    def __init__(self, a, b, c):
        self.a, self.b, self.c = a, b, c

    def aire(self):
        s = self.perimetre() / 2
        return math.sqrt(s*(s-self.a)*(s-self.b)*(s-self.c))

    def perimetre(self):
        return self.a + self.b + self.c


# POLYMORPHISME     meme appel, comportements differents
formes = [
    Cercle(5),
    Rectangle(4, 6),
    Triangle(3, 4, 5),
    Cercle(2),
    Rectangle(10, 3),
]

print("=== Inventaire des formes ===")
for forme in formes:
    forme.decrire()     # chaque forme calcule sa propre aire !

# Calculer l'aire totale
total = sum(f.aire() for f in formes)
```

```python
print(f"\nAire totale : {total:.2f} cm ")
```

Les classes abstraites

Une **classe abstraite** définit une interface — elle impose que les classes filles implémentent certaines méthodes.

Classes abstraites avec ABC

```python
from abc import ABC, abstractmethod

class Animal(ABC):
    """Classe abstraite ne peut pas etre instanciee
       directement."""

    def __init__(self, nom):
        self.nom = nom

    @abstractmethod
    def parler(self):
        """Chaque animal doit implementer parler()."""
        pass

    @abstractmethod
    def se_deplacer(self):
        """Chaque animal doit implementer se_deplacer()."""
        pass

    # Methode concrete heritee telle quelle
    def respirer(self):
        print(f"{self.nom} respire.")


class Chien(Animal):
    def parler(self):
        print(f"{self.nom} : Wouf !")

    def se_deplacer(self):
        print(f"{self.nom} court sur 4 pattes.")


class Oiseau(Animal):
    def parler(self):
        print(f"{self.nom} : Tweet !")

    def se_deplacer(self):
        print(f"{self.nom} vole dans les airs.")
```

```python
class Poisson(Animal):
    def parler(self):
        print(f"{self.nom} : ...")

    def se_deplacer(self):
        print(f"{self.nom} nage dans l'eau.")


# Instancier une classe abstraite      TypeError
# a = Animal("Generique")

# Instancier les classes concretes
animaux = [
    Chien("Rex"),
    Oiseau("Tweety"),
    Poisson("Nemo"),
]

for animal in animaux:
    animal.parler()
    animal.se_deplacer()
    animal.respirer()    # methode heritee
    print()
```

Ce qui m'a résisté

La différence entre lever `NotImplementedError` manuellement et utiliser `@abstractmethod` est significative. Avec `NotImplementedError`, la classe peut être instanciée — l'erreur n'arrive qu'à l'appel. Avec `@abstractmethod`, Python **refuse l'instanciation** dès la création si une méthode abstraite n'est pas implémentée. C'est un contrat au niveau de la classe, pas de la méthode. Cette distinction m'a montré à quel point Python peut être strict ou souple selon nos besoins.

L'héritage multiple

Python supporte l'**héritage multiple** — une classe peut hériter de plusieurs classes parentes.

Héritage multiple et MRO

```python
class Volant:
    def voler(self):
        print(f"{self.nom} vole !")

    def decoller(self):
        print(f"{self.nom} d colle.")
```

```python
class Nageur:
    def nager(self):
        print(f"{self.nom} nage !")

    def plonger(self):
        print(f"{self.nom} plonge.")


class Coureur:
    def courir(self):
        print(f"{self.nom} court !")


# Canard herite de trois classes
class Canard(Volant, Nageur, Coureur):
    def __init__(self, nom):
        self.nom = nom

    def tout_faire(self):
        self.voler()
        self.nager()
        self.courir()


donald = Canard("Donald")
donald.tout_faire()
donald.decoller()
donald.plonger()

# MRO      Method Resolution Order
# Python r sout les m thodes dans cet ordre precis
print(Canard.__mro__)
# (<class 'Canard'>, <class 'Volant'>,
#  <class 'Nageur'>, <class 'Coureur'>, <class 'object'>)

# Consulter le MRO lisiblement
for classe in Canard.__mro__:
    print(f"      {classe.__name__}")
```

Les propriétés — @property

Le décorateur `@property` permet de créer des **attributs calculés** et de contrôler l'accès aux attributs.

@property — getters, setters, deleters

```python
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius    # attribut "protege"
```

```python
    # Getter      lecture
    @property
    def celsius(self):
        return self._celsius

    # Setter      ecriture avec validation
    @celsius.setter
    def celsius(self, valeur):
        if valeur < -273.15:
            raise ValueError("Temperature en dessous du zero
               absolu !")
        self._celsius = valeur

    # Propriete calculee      Fahrenheit
    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, valeur):
        self.celsius = (valeur - 32) * 5/9

    # Propriete calculee      Kelvin
    @property
    def kelvin(self):
        return self._celsius + 273.15

    def __str__(self):
        return (f"{self._celsius:.1f} C  | "
                f"{self.fahrenheit:.1f} F  | "
                f"{self.kelvin:.1f}K")


t = Temperature(100)
print(t)   # 100.0 C  | 212.0 F  | 373.1K

t.celsius = 0
print(t)   # 0.0 C  | 32.0 F  | 273.1K

t.fahrenheit = 98.6
print(t)   # 37.0 C  | 98.6 F  | 310.1K

#  Validation en action
try:
    t.celsius = -300
except ValueError as e:
    print(f"Erreur : {e}")
```

@staticmethod et @classmethod

Méthodes statiques et de classe

```python
class Date:
    def __init__(self, jour, mois, annee):
        self.jour  = jour
        self.mois  = mois
        self.annee = annee

    # Methode d'instance     accede a self
    def afficher(self):
        return f"{self.jour:02d}/{self.mois:02d}/{self.annee}"

    # @classmethod     recoit la classe (cls), pas l'instance
    # Utilise pour creer des constructeurs alternatifs
    @classmethod
    def depuis_chaine(cls, chaine):
        """Creer une Date depuis '14/03/2026'."""
        jour, mois, annee = map(int, chaine.split("/"))
        return cls(jour, mois, annee)

    @classmethod
    def aujourd_hui(cls):
        """Creer une Date representant aujourd'hui."""
        from datetime import date
        d = date.today()
        return cls(d.day, d.month, d.year)

    # @staticmethod     ni self ni cls
    # Fonction utilitaire liee    la classe mais independante
    @staticmethod
    def est_annee_bissextile(annee):
        return (annee % 4 == 0 and annee % 100 != 0) or (annee
            % 400 == 0)

    @staticmethod
    def valider_date(jour, mois, annee):
        return 1 <= mois <= 12 and 1 <= jour <= 31 and annee >
            0


# Utilisation
d1 = Date(14, 3, 2026)
d2 = Date.depuis_chaine("25/02/2026")
d3 = Date.aujourd_hui()

print(d1.afficher())   # 14/03/2026
print(d2.afficher())   # 25/02/2026
print(d3.afficher())   # date du jour
```

```
# Methodes statiques      pas d'instance necessaire
print(Date.est_annee_bissextile(2024))    # True
print(Date.est_annee_bissextile(2023))    # False
print(Date.valider_date(31, 13, 2024))    # False
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `super()` suit le MRO — pas juste "la classe parente". Je pensais que `super()` appelait toujours la classe directement parente. En réalité, il suit le **Method Resolution Order** — un algorithme C3 linéarisé qui garantit que chaque classe n'est visitée qu'une seule fois dans l'héritage multiple. Comprendre le MRO m'a permis de concevoir des hiérarchies de classes sans ambiguïté, même complexes.

2. `@property` transforme une méthode en attribut. Écrire `objet.celsius` au lieu de `objet.get_celsius()` semble anodin. Mais c'est une différence philosophique majeure — l'interface reste propre, la validation et le calcul sont cachés. Le **principe d'encapsulation** devient concret : l'utilisateur de la classe ne sait pas — et n'a pas besoin de savoir — si `celsius` est stocké ou calculé.

3. `@classmethod` comme constructeur alternatif. Créer plusieurs façons d'instancier une classe — depuis une chaîne, depuis un fichier, depuis une API — sans toucher à `__init__` est une technique de design élégante. `Date.depuis_chaine("15/03/2024")` est bien plus expressif que `Date(*"15/03/2024".split("/"))`. C'est l'esprit Python : rendre le code lisible comme de la prose.

Résumé du Chapitre 16

Ce que j'ai appris dans ce chapitre :

- **Héritage** : `class Enfant(Parent)` — réutiliser et spécialiser.
- `super()` — appeler constructeurs et méthodes de la classe parente.
- **Override** — redéfinir une méthode dans la classe enfant.
- **Polymorphisme** — même interface, comportements multiples.
- **Classes abstraites** (`ABC`, `@abstractmethod`) — imposer un contrat.
- **Héritage multiple** et **MRO** — résolution d'ordre des méthodes.
- `@property` — attributs calculés avec getter, setter, deleter.
- `@classmethod` — constructeurs alternatifs recevant `cls`.

- `@staticmethod` — fonctions utilitaires liées à la classe.

"L'héritage, c'est la réutilisation. Le polymorphisme, c'est la flexibilité. Ensemble, ils permettent d'écrire du code qui s'adapte sans se répéter."